Ex.No : 10 **Heart Rate Monitoring and ECG Arrhythmia Classification using**
Date : **Machine Learning**

Problem Statement:

Electrocardiogram (ECG) signals provide important information about the electrical activity of the heart. Changes in waveform shape, beat timing and heart rate may be associated with different cardiac conditions such as bradycardia, tachycardia and arrhythmia. Manual inspection of ECG traces is time-consuming, subjective and requires specialist knowledge, particularly when many recordings must be reviewed.

There is, therefore, a need for an automated and accessible system that can preprocess an ECG image or signal, identify heartbeat peaks, calculate heart-rate-related measurements and support the classification of cardiac rhythm patterns with a measurable confidence score. This project designs and implements a Heart Rate Monitoring and ECG Classification application using classical image processing (OpenCV), signal processing (SciPy) and a Random Forest classifier in Scikit-Learn. The workflow covers ECG image preprocessing, waveform reconstruction, R-peak detection, RR-interval calculation, heart rate and heart-rate-variability (HRV) feature extraction, and machine-learning-based diagnostic assessment. The application displays the intermediate processing stages, numerical measurements, diagnostic confidence and model evaluation visualisations (Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC curves and Feature Importance).

Objectives:

1. To organise ECG sample images for four rhythm classes: Normal Sinus Rhythm, Sinus Bradycardia, Sinus Tachycardia and Cardiac Arrhythmia.
2. To preprocess ECG images using grayscale conversion, CLAHE contrast enhancement, denoising, Otsu binarisation and morphological processing.
3. To reconstruct a one-dimensional ECG waveform from the processed trace and detect R-peaks in the signal.
4. To calculate RR intervals, instantaneous heart rate (BPM) and selected HRV / statistical features.
5. To train and use a Random Forest classifier for ECG rhythm assessment.
6. To display the predicted rhythm, class probability distribution and confidence in a web dashboard.
7. To evaluate the model using Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC curves and Feature Importance.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code
- Language: Python 3.x

- Image Processing: OpenCV (opencv-python), NumPy, Pillow
- Data Processing and Analysis: Pandas, NumPy, SciPy
- Machine Learning: scikit-learn (RandomForestClassifier, train_test_split, metrics)
- Data Visualisation: Matplotlib, Seaborn
- Model Serialisation: Joblib
- Application: Python web application / dashboard
- Input Data: ECG sample images and processed ECG signals

Project Directory Structure:

The project is organised into separate modules for preprocessing, feature extraction and model training, together with sample ECG images, trained-model files and the application interface, as shown below.

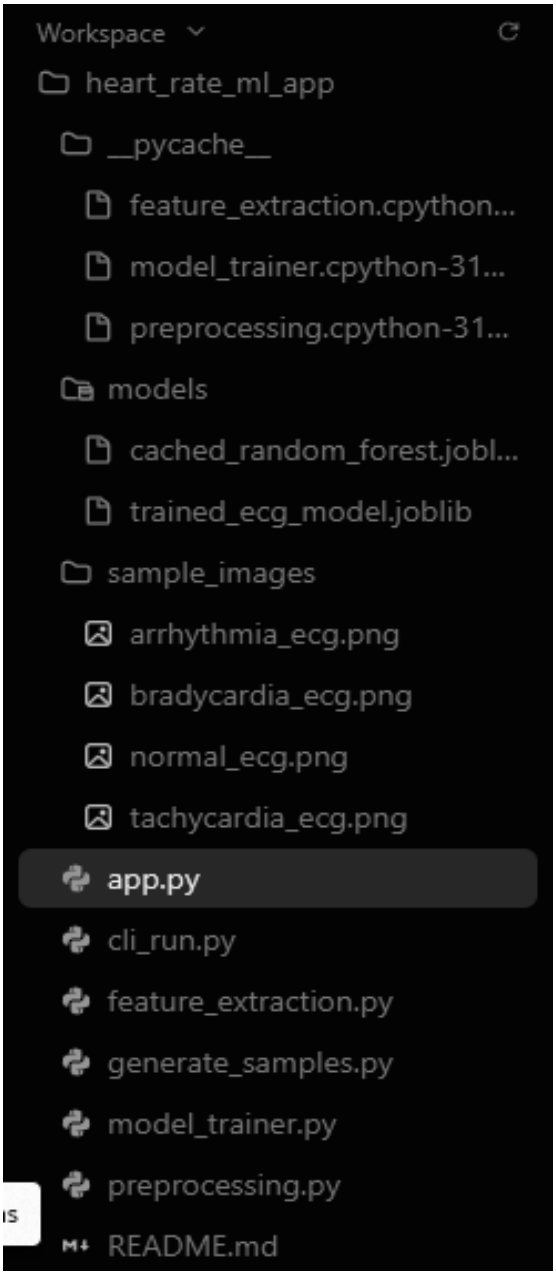


Fig 1: Project directory structure of heart_rate_ml_app in Visual Studio Code

Procedure:

1. Load the ECG sample images from the sample_images directory.
2. Read each input image and convert it to a single-channel grayscale image.
3. Improve local contrast using Contrast Limited Adaptive Histogram Equalization (CLAHE).
4. Apply denoising to reduce unwanted image noise while retaining ECG trace details.
5. Use Otsu thresholding to obtain a binary representation of the ECG trace.
6. Apply morphological processing to highlight the ECG trace and obtain a skeleton representation.
7. Reconstruct a continuous one-dimensional ECG waveform from the processed trace.
8. Detect R-peaks and calculate the time difference between consecutive peaks (RR intervals).
9. Calculate heart rate in beats per minute (BPM) and extract statistical and HRV features.
10. Split the feature dataset into training and test sets using train_test_split(stratify=y, random_state=42) and train the Random Forest model.
11. Evaluate the model on the held-out test set (Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC curves, Feature Importance) and save it with Joblib.
12. Pass the extracted features of a new ECG image to the trained model and display the predicted rhythm, confidence and class probabilities.

Implementation Code:

Representative implementation of the key modules used in the project. The exact source files were not included with the screenshots; the code below reflects the processing and model workflow shown by the application.

1. ECG Image Loading and Preprocessing (preprocessing.py)

```
import cv2

import numpy as np

def preprocess_ecg(image_path):

    # Read the original ECG image

    image = cv2.imread(image_path)

    if image is None:

        raise ValueError("Unable to read ECG image")

    # Convert RGB image to grayscale

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Enhance local contrast (CLAHE)

    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))

    enhanced = clahe.apply(gray)
```

```
    # Reduce image noise
    denoised = cv2.GaussianBlur(enhanced, (3, 3), 0)
    # Otsu binary thresholding
    _, binary = cv2.threshold(
        denoised, 0, 255,
        cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    return image, gray, enhanced, denoised, binary
```

2. Morphological Processing and Skeleton Trace (preprocessing.py)

```
def extract_skeleton(binary):
    # Morphological skeletonization
    skeleton = np.zeros(binary.shape, np.uint8)
    element = cv2.getStructuringElement(cv2.MORPH_CROSS, (3, 3))
    work = binary.copy()
    while cv2.countNonZero(work) > 0:
        opened = cv2.morphologyEx(work, cv2.MORPH_OPEN, element)
        residue = cv2.subtract(work, opened)
        skeleton = cv2.bitwise_or(skeleton, residue)
        work = cv2.erode(work, element)
    return skeleton
```

3. Waveform Reconstruction, R-Peak Detection and HRV Extraction (feature_extraction.py)

```
import numpy as np
from scipy.signal import find_peaksdef extract_heart_features(signal, sampling_rate):
    signal = np.asarray(signal, dtype=float)
    signal = (signal - signal.mean()) / (signal.std() + 1e-8)
    # Detect prominent R-peaks (minimum 0.30 s apart)
    min_distance = int(0.30 * sampling_rate)
    peaks, _ = find_peaks(signal, distance=min_distance, prominence=0.8)
    # RR intervals in seconds and milliseconds
    rr_sec = np.diff(peaks) / sampling_rate
    rr_ms = rr_sec * 1000.0
    mean_rr = np.mean(rr_ms) if len(rr_ms) else 0.0
```

```
    heart_rate = 60000.0 / mean_rr if mean_rr > 0 else 0.0
    # HRV time-domain features
    sdnn = np.std(rr_ms, ddof=1) if len(rr_ms) > 1 else 0.0
    rmssd = (np.sqrt(np.mean(np.diff(rr_ms) ** 2))
            if len(rr_ms) > 1 else 0.0)
    pnn50 = (np.mean(np.abs(np.diff(rr_ms)) > 50) * 100
            if len(rr_ms) > 1 else 0.0)
    return peaks, rr_ms, heart_rate, mean_rr, sdnn, rmssd, pnn50
```

4. Additional Statistical Features (feature_extraction.py)

```
from scipy.stats import skew, kurtosis
def entropy_from_histogram(x, bins=20):
    hist, _ = np.histogram(x, bins=bins)
    p = hist[hist > 0] / hist.sum()
    return -np.sum(p * np.log2(p))
def statistical_features(signal):
    x = np.asarray(signal, dtype=float)
    x = (x - np.mean(x)) / (np.std(x) + 1e-8)
    return {
        "signal_mean": float(np.mean(x)),
        "signal_std": float(np.std(x)),
        "signal_skew": float(skew(x)),
        "signal_kurtosis": float(kurtosis(x)),
        "signal_energy": float(np.sum(x ** 2)),
        "signal_p2p": float(np.ptp(x)),
        "signal_entropy": float(entropy_from_histogram(x))
    }
```

5. Random Forest Model Training and Evaluation (model_trainer.py)

```
import joblib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
```

```
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)
# Load the feature dataset
data = pd.read_csv("ecg_features.csv")
X = data.drop(columns=["label"])
y = data["label"]
# Split data into training and testing partitions
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
# Train the Random Forest classifier
model = RandomForestClassifier(
    n_estimators=100, random_state=42, class_weight="balanced"
)
model.fit(X_train, y_train)
# Evaluate the trained model
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
# Save the trained model
joblib.dump(model, "models/trained_ecg_model.joblib")
```

6. Prediction and Class Probability (app.py)

```
def predict_rhythm(model, feature_values):
    features = pd.DataFrame([feature_values])
    predicted_class = model.predict(features)[0]    probabilities = model.predict_proba(features)[0]
    probability_map = dict(zip(model.classes_, probabilities))
    return predicted_class, probability_map
```

Output (Step-by-Step Screenshots):

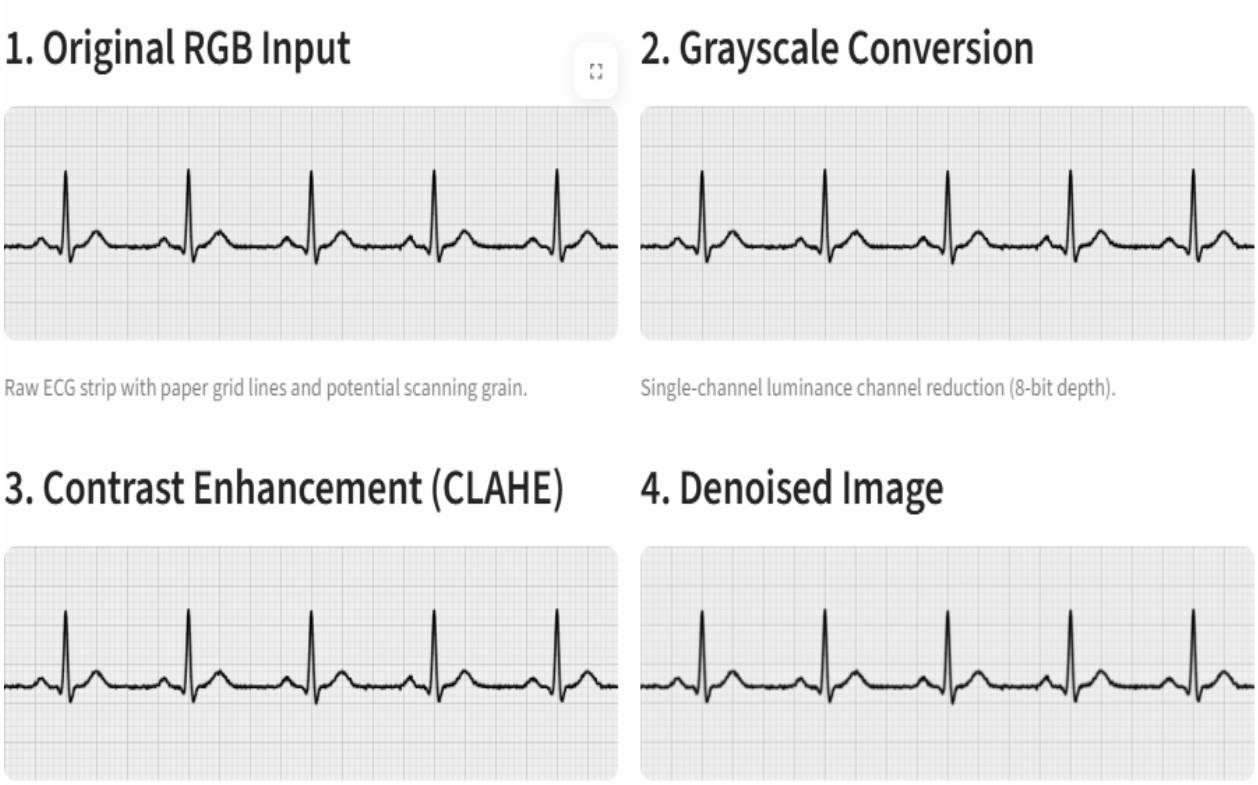


Fig 2: Step 1 — ECG Preprocessing: original RGB input, grayscale conversion, CLAHE contrast enhancement and denoised image.

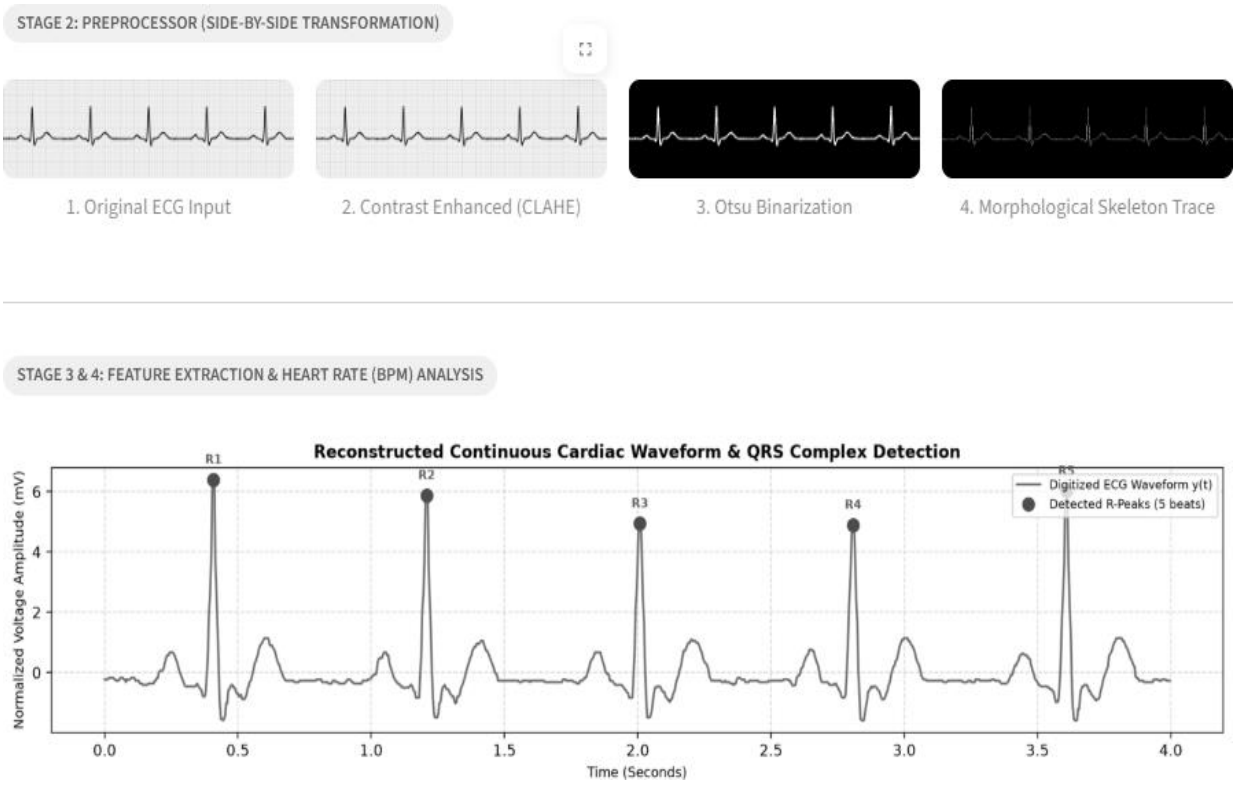


Fig 3: Step 2 — Application dashboard: preprocessing stages (original, CLAHE, Otsu binarisation, skeleton trace) and reconstructed waveform with R-peak detection

1D Waveform Reconstruction & HRV Extraction

Detailed breakdown of peak detection, RR interval calculation, and Heart Rate Variability (HRV) feature engineering.

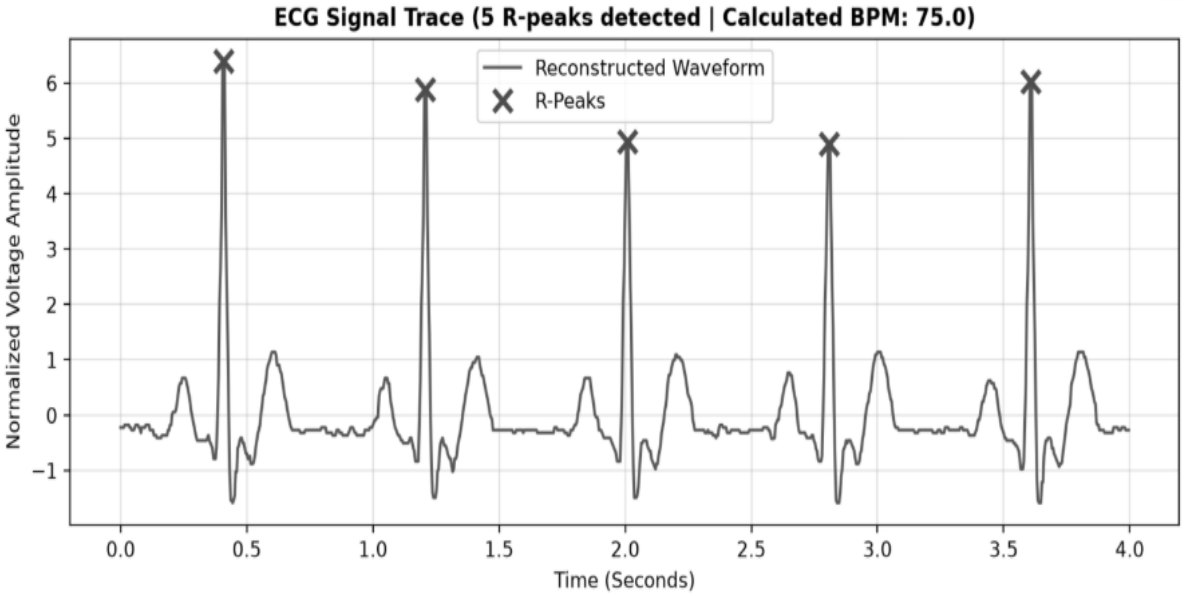


Fig 4: Step 3 — Reconstructed ECG waveform with five detected R-peaks (R1 to R5).

Beat-to-Beat RR Interval Timeline

	Beat Transition	RR Interval (seconds)	RR Interval (milliseconds)	Instantaneous Heart Rate (BPM)
0	Beat 1 ➔ Beat 2	0.800	800.0	75.0
1	Beat 2 ➔ Beat 3	0.800	800.0	75.0
2	Beat 3 ➔ Beat 4	0.800	800.0	75.0
3	Beat 4 ➔ Beat 5	0.800	800.0	75.0

Calculated Statistical Moments & Non-Linear Features

	Value
signal_mean	0.0000
signal_std	1.0000
signal_skew	3.3950
signal_kurtosis	15.6228
signal_energy	1.0000
signal_p2p	7.9839
signal_entropy	2.9861

Fig 5: Step 4 — Beat-to-beat RR interval timeline and calculated statistical / non-linear features

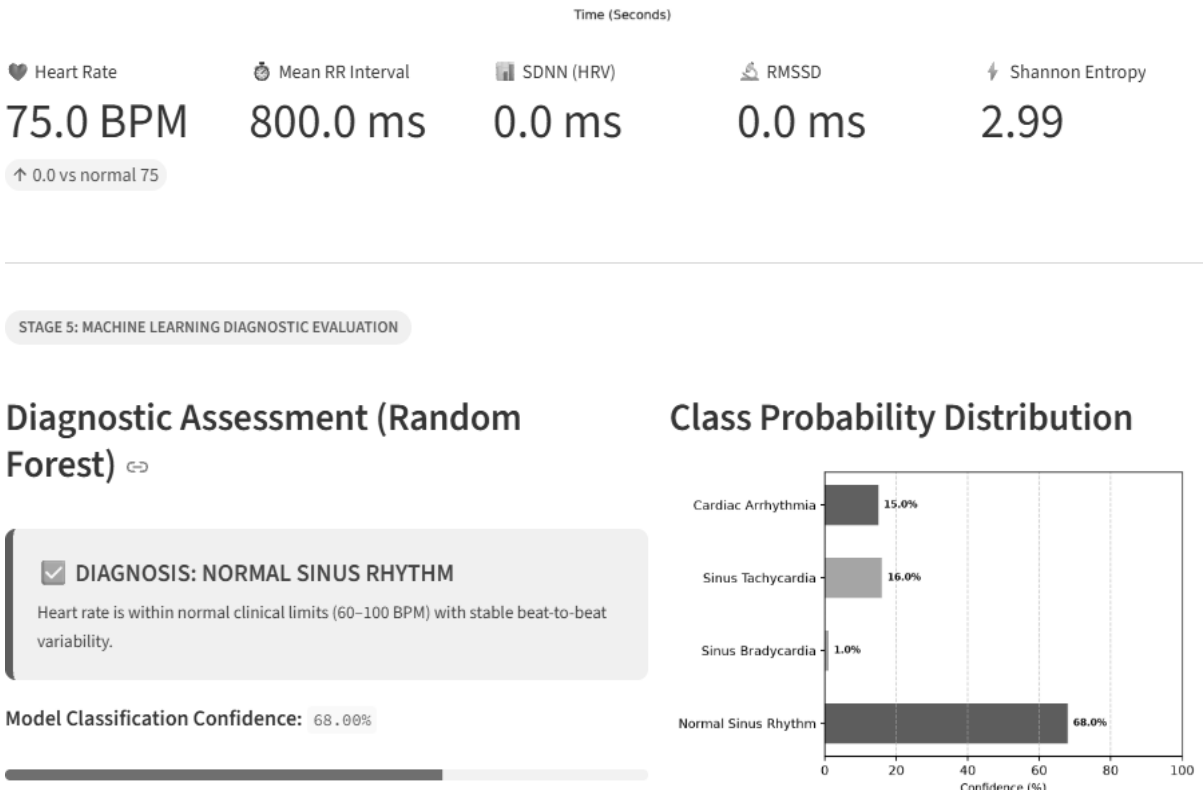


Fig 6: Step 5 — Heart-rate metrics and Random Forest diagnosis with class probability distribution

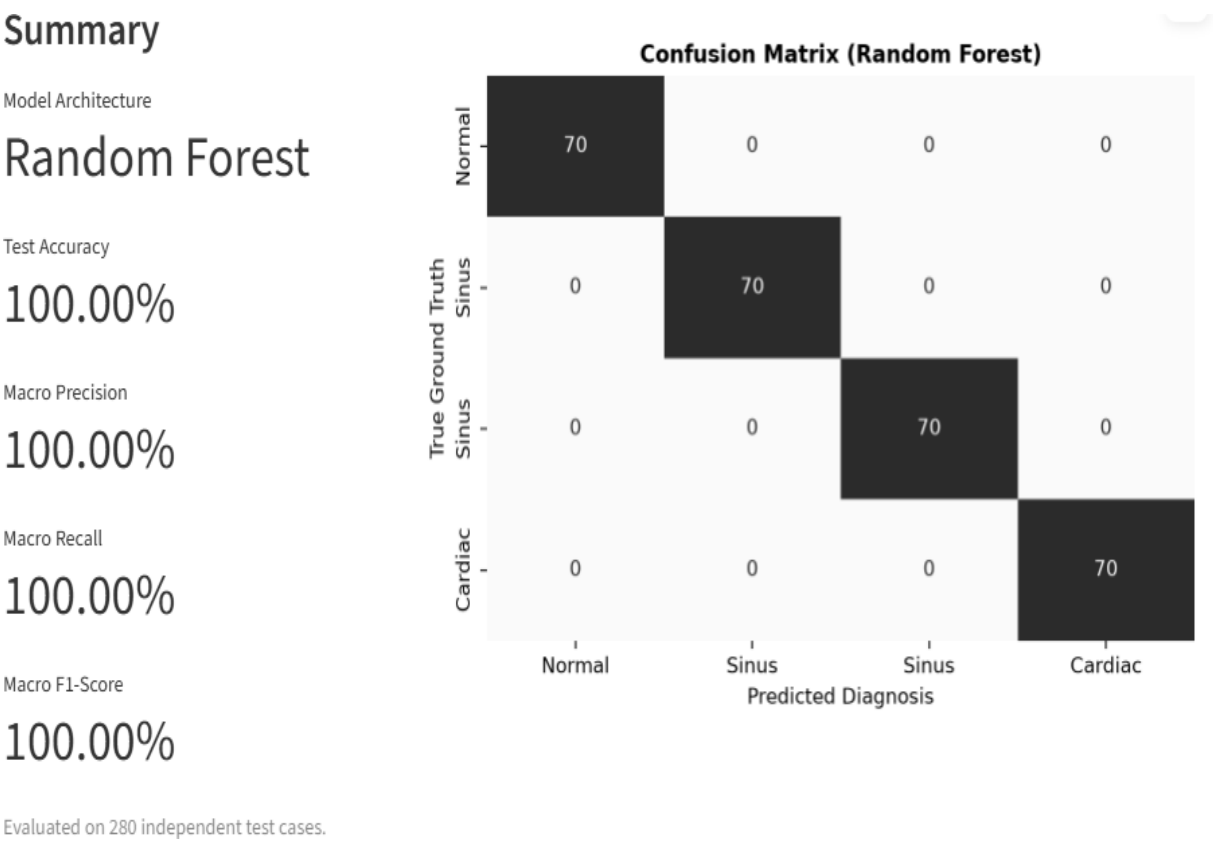


Fig 7: Step 6 — Performance Evaluation: Random Forest summary metrics and confusion matrix

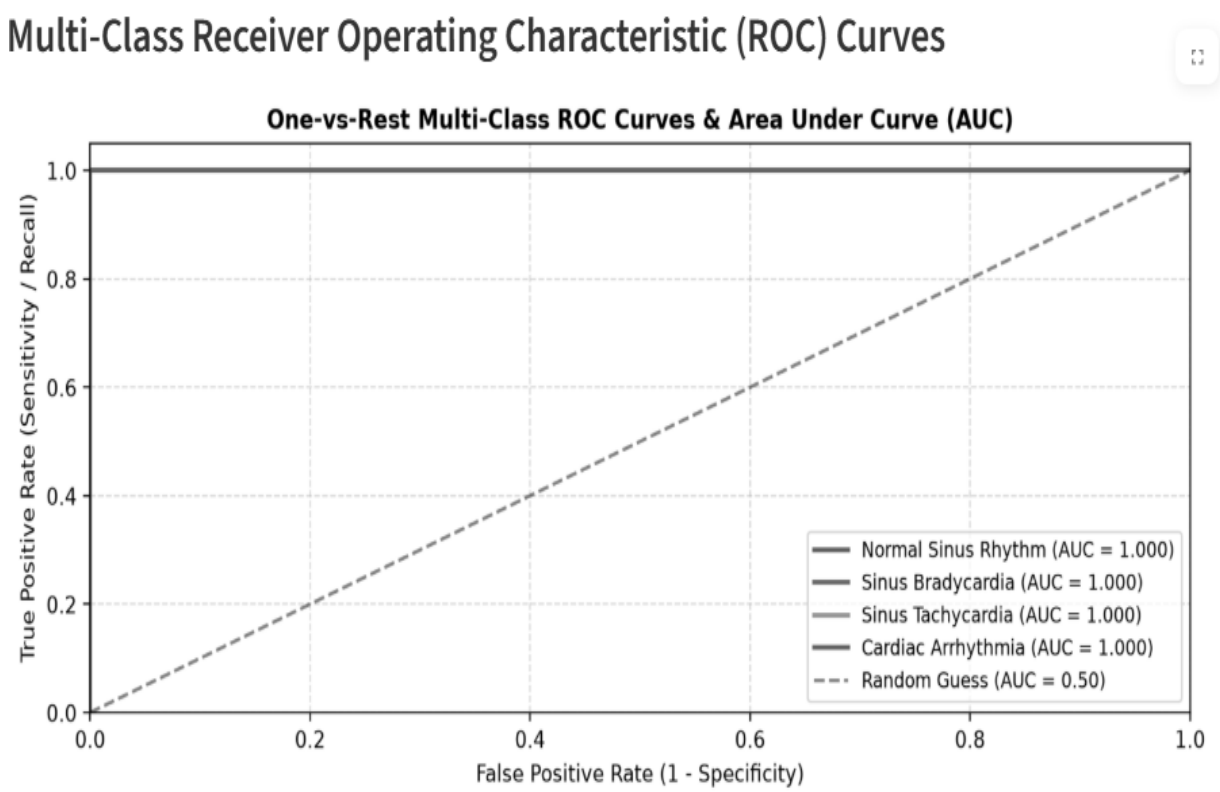


Fig 8: Step 6 — Multi-class Receiver Operating Characteristic (ROC) curves with AUC for all four rhythm classes.

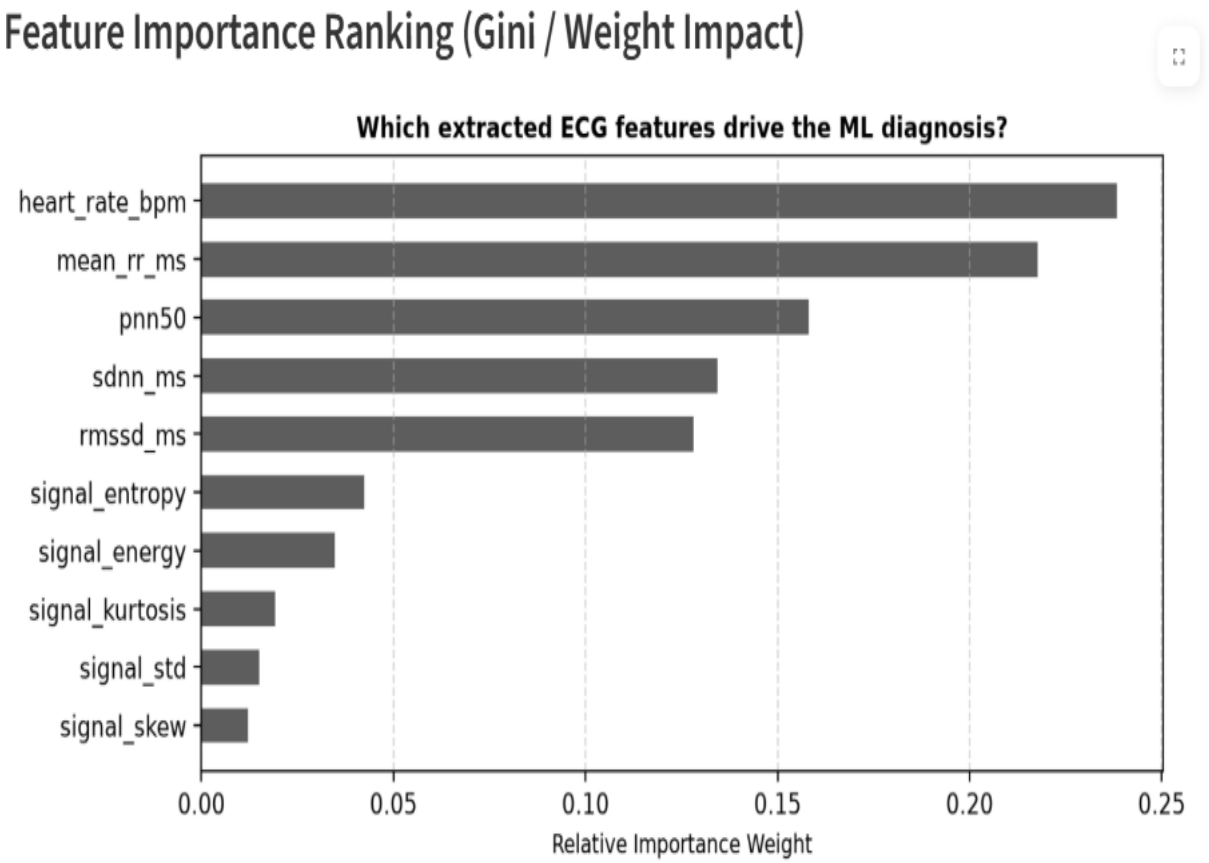


Fig 9: Step 6 — Feature importance ranking (Gini / weight impact)

Result:

An end-to-end Heart Rate Monitoring and ECG Classification application was designed and implemented using OpenCV image processing, SciPy-based R-peak detection, HRV feature extraction and a Scikit-Learn Random Forest classifier. ECG images were converted to grayscale, enhanced with CLAHE, denoised, binarised with Otsu thresholding and reduced to a skeleton trace, from which a one-dimensional waveform was reconstructed. For the sample ECG, five R-peaks were detected, the mean RR interval was 800.0 ms and the calculated heart rate was 75.0 BPM, with Normal Sinus Rhythm predicted at 68.00% confidence. The Random Forest model achieved 100.00% test Accuracy, Precision, Recall and F1-Score on 280 independent test cases, a diagonal confusion matrix (70 correct predictions per class) and an AUC of 1.000 for all four rhythm classes. Thus, the project demonstrates an integrated workflow for ECG image preprocessing, heart-rate and HRV feature extraction, Random Forest-based rhythm classification and visual presentation of model outputs. The displayed metrics describe the supplied application example and should be interpreted in the context of the dataset and evaluation procedure used by the project.